Q3.1: Add Contact Information to Staff (MOST COMMON TIER 3)

Question: Add phoneNumber (String) and email (String) fields to the HotelStaff class with appropriate getters and setters. Update the console input process to collect this information when adding staff members. Modify the staff table to display email addresses in a new column.

SOLUTION STEPS:

STEP 1: Update HotelStaff.java - Add Fields

FIND THE EXISTING FIELDS:

```
private String name;
private String surname;
private LocalDate dateOfBirth;
private String staffID;
```

ADD AFTER THEM:

```
private String phoneNumber;
private String email;
```

STEP 2: Add Getter and Setter Methods to HotelStaff.java

ADD AFTER EXISTING GETTER/SETTER METHODS:

```
// Phone Number getter and setter
public void setPhoneNumber(String phoneNumber) {
   this.phoneNumber = phoneNumber;
}

public String getPhoneNumber() {
   return phoneNumber;
}

// Email getter and setter
public void setEmail(String email) {
   this.email = email;
}
```

```
public String getEmail() {
   return email;
}
```

STEP 3: Update WestminsterHotelManager.java - addHotelStaff() Method

FIND THIS SECTION (after date input):

```
}

// Check if the staff is a manager or a guest
switch (choiceStaff) {
```

ADD BEFORE THE SWITCH STATEMENT:

```
}

// Collect contact information
System.out.println("Enter the phone number");
String phoneNumber = s.nextLine();

System.out.println("Enter the email address");
String email = s.nextLine();

// Check if the staff is a manager or a guest
switch (choiceStaff) {
```

STEP 4: Update All Cases in Switch Statement

FIND EACH CASE (Manager, HouseKeeper, etc.) and ADD contact info:

For Manager case:

```
case 1:
   // It is a manager
   System.out.println("Enter the license number");
```

```
    String licenseNum = s.nextLine();

    Manager manager = new Manager(name, surname);
    manager.setLicenseNumber(licenseNum);
    manager.setDateOfBirth(date);
    manager.setStaffID(staffID);
    manager.setPhoneNumber(phoneNumber);  // ADD THIS
    manager.setEmail(email);           // ADD THIS
    this.addStaffToList(manager);
    break;
```

For HouseKeeper case:

```
case 2:
    // It is a HouseKeeper
    System.out.println("Enter the years of experience");
    int yearsOfExperience = s.nextInt();
    s.nextLine();

    HouseKeeper housekeeper = new HouseKeeper(name, surname);
    housekeeper.setYearsOfExperience(yearsOfExperience);
    housekeeper.setDateOfBirth(date);
    housekeeper.setStaffID(staffID);
    housekeeper.setPhoneNumber(phoneNumber);  // ADD THIS
    housekeeper.setEmail(email);           // ADD THIS
    this.addStaffToList(housekeeper);
    break;
```

Apply same pattern to any other staff types (Receptionist, Chef, etc.)

STEP 5: Update HotelTableModel.java - Add Email Column

FIND THIS LINE:

private String[] columnNames = {"Name", "Surname", "Date of Birth", "Role"};

REPLACE WITH:

private String[] columnNames = {"Name", "Surname", "Date of Birth", "Role", "Email"};

FIND THE getValueAt() METHOD and ADD:

} else if (columnIndex == 4) {

  temp = list.get(rowIndex).getEmail();

}

Q3.2: Add Check-in/Check-out Dates to Guest (DATE MANAGEMENT)

Question: Add checkInDate and checkOutDate (LocalDate) fields to the Guest class. Update the guest input process to collect these dates in dd/MM/yyyy format. Modify the toString() method to display the check-in and check-out dates. Add validation to ensure check-out date is after check-in date.

SOLUTION STEPS:

STEP 1: Update Guest.java - Add Fields

FIND THE EXISTING FIELDS:

private String name;

private String surname;

private int roomNumber;

private int nightsStayed;

ADD AFTER THEM:

private LocalDate checkInDate;

private LocalDate checkOutDate;

STEP 2: Add Required Import to Guest.java

ADD AT THE TOP:

import java.time.LocalDate;

import java.time.format.DateTimeFormatter;

STEP 3: Add Getter/Setter Methods to Guest.java

ADD AFTER EXISTING GETTERS/SETTERS:

// Check-in date getter and setter

public void setCheckInDate(LocalDate checkInDate) {

```
    this.checkInDate = checkInDate;
}

public LocalDate getCheckInDate() {
    return checkInDate;
}

// Check-out date getter and setter
public void setCheckOutDate(LocalDate checkOutDate) {
    this.checkOutDate = checkOutDate;
}

public LocalDate getCheckOutDate() {
    return checkOutDate;
}

// Utility method to get formatted date string
public String getFormattedCheckInDate() {
    if (checkInDate != null) {
        DateTimeFormatter formatter = DateTimeFormatter.ofPattern("dd/MM/yyyy");
        return checkInDate.format(formatter);
    }
    return "Not set";
}

public String getFormattedCheckOutDate() {
    if (checkOutDate != null) {
        DateTimeFormatter formatter = DateTimeFormatter.ofPattern("dd/MM/yyyy");
```

```
        return checkOutDate.format(formatter);
    }
    return "Not set";
}
```

STEP 4: Update toString() Method in Guest.java

FIND THE CURRENT toString():

```
@Override
public String toString() {
    return name + " " + surname + " - Room number: " + roomNumber + ", Nights stayed: " +
nightsStayed;
}
```

REPLACE WITH:

```
@Override
public String toString() {
    return name + " " + surname + " - Room: " + roomNumber +
            ", Nights: " + nightsStayed +
            ", Check-in: " + getFormattedCheckInDate() +
            ", Check-out: " + getFormattedCheckOutDate();
}
```

STEP 5: Update WestminsterHotelManager.java - addGuest() Method

ADD REQUIRED IMPORT:

```
import java.time.format.DateTimeParseException;
```

FIND THE addGuest() METHOD and REPLACE WITH:

```
@Override
public void addGuest() {
    Scanner s = new Scanner(System.in);


    System.out.println("Enter the first name");
    String name = s.nextLine();
```

```
System.out.println("Enter the last name");
String surname = s.nextLine();

System.out.println("Enter the room number");
int roomNum = s.nextInt();
s.nextLine();

System.out.println("Enter the number of nights stayed");
int nightsStayed = s.nextInt();
s.nextLine();

// Get check-in date
System.out.println("Enter the check-in date (dd/MM/yyyy format)");
LocalDate checkInDate = null;
boolean validCheckIn = false;
while (!validCheckIn) {
   String checkInInput = s.nextLine();
   DateTimeFormatter formatter = DateTimeFormatter.ofPattern("dd/MM/yyyy");
   try {
      checkInDate = LocalDate.parse(checkInInput, formatter);
      validCheckIn = true;
   } catch (DateTimeParseException e) {
      System.out.println("Invalid date format! Please use dd/MM/yyyy:");
   }
}

// Get check-out date
```

```
    System.out.println("Enter the check-out date (dd/MM/yyyy format)");
    LocalDate checkOutDate = null;
    boolean validCheckOut = false;
    while (!validCheckOut) {
        String checkOutInput = s.nextLine();
        DateTimeFormatter formatter = DateTimeFormatter.ofPattern("dd/MM/yyyy");
        try {
            checkOutDate = LocalDate.parse(checkOutInput, formatter);

            // Validate that check-out is after check-in
            if (checkOutDate.isAfter(checkInDate)) {
                validCheckOut = true;
            } else {
                System.out.println("Check-out date must be after check-in date! Please enter a
valid check-out date:");
            }
        } catch (DateTimeParseException e) {
            System.out.println("Invalid date format! Please use dd/MM/yyyy:");
        }
    }

    Guest guest = new Guest(name, surname);
    guest.setRoomNumber(roomNum);
    guest.setNightsStayed(nightsStayed);
    guest.setCheckInDate(checkInDate);
    guest.setCheckOutDate(checkOutDate);

    hotelGuestList.add(guest);
}
```

Q3.9: Add Staff Search by Name (BASIC SEARCH FUNCTIONALITY)

Question: Add a searchStaffByName(String name) method that returns a list of staff members whose first name or surname contains the search term (case-insensitive). Create a search GUI with a text field and "Search" button that displays results in a popup dialog or separate window.

SOLUTION STEPS:

STEP 1: Add Search Method to WestminsterHotelManager.java

```
// Search method to find staff by name
public ArrayList<HotelStaff> searchStaffByName(String searchTerm) {
  ArrayList<HotelStaff> searchResults = new ArrayList<>();
  String searchLower = searchTerm.toLowerCase();

  for (HotelStaff staff : hotelStaffList) {
    String firstName = staff.getName().toLowerCase();
    String lastName = staff.getSurname().toLowerCase();

    if (firstName.contains(searchLower) || lastName.contains(searchLower)) {
      searchResults.add(staff);
    }
  }

  return searchResults;
}
```

STEP 2: Add Console Search Option

UPDATE runMenu() METHOD - ADD TO MENU DISPLAY:

```
System.out.println("To Search for staff by name, press 8");
```

ADD TO SWITCH STATEMENT:

```
case 8:
  this.searchStaffConsole();
  break;
```

STEP 3: Add Console Search Method

```
// Console-based staff search
public void searchStaffConsole() {
   Scanner s = new Scanner(System.in);
   System.out.println("Enter name to search for:");
   String searchTerm = s.nextLine();

   ArrayList<HotelStaff> results = searchStaffByName(searchTerm);

   if (results.isEmpty()) {
      System.out.println("No staff members found with name containing: " + searchTerm);
   } else {
      System.out.println("\nSearch Results (" + results.size() + " found):");
      System.out.println("====================================");
      for (HotelStaff staff : results) {
         System.out.println(staff.toString());
      }
   }
}
```

STEP 4: Add GUI Search to HotelTableGUI.java

ADD REQUIRED IMPORTS:

```
import javax.swing.JTextField;
import java.awt.FlowLayout;
```

UPDATE THE BUTTON PANEL SECTION: FIND:

```
// Create a panel for multiple buttons
JPanel buttonPanel = new JPanel();
```

REPLACE WITH:

```
// Create a panel for buttons and search
```

```
JPanel buttonPanel = new JPanel(new FlowLayout());

// Add search components
JTextField searchField = new JTextField(15);
searchField.setToolTipText("Enter name to search");
JButton searchButton = new JButton("Search");

// Search functionality
searchButton.addActionListener(e -> {
  String searchTerm = searchField.getText().trim();
  if (!searchTerm.isEmpty()) {
    // This requires access to WestminsterHotelManager
    // For simplicity, we'll show a basic filter approach
    StringBuilder results = new StringBuilder("Search Results for: " + searchTerm + "\n\n");
    boolean found = false;

    for (HotelStaff staff : list) {
      String firstName = staff.getName().toLowerCase();
      String lastName = staff.getSurname().toLowerCase();
      String search = searchTerm.toLowerCase();

      if (firstName.contains(search) || lastName.contains(search)) {
        results.append(staff.toString()).append("\n");
        found = true;
      }
    }

    if (!found) {
```

```
        results.append("No staff members found with name containing:
").append(searchTerm);
    }

    JOptionPane.showMessageDialog(this, results.toString(), "Search Results",
JOptionPane.INFORMATION_MESSAGE);
    searchField.setText(""); // Clear search field
  } else {
    JOptionPane.showMessageDialog(this, "Please enter a search term!", "Empty Search",
JOptionPane.WARNING_MESSAGE);
  }
});

// Add search components to panel
buttonPanel.add(searchField);
buttonPanel.add(searchButton);
```

Q3.5: Add Edit Staff Functionality (ADVANCED GUI)

Question: Add an "Edit Staff" button to the HotelTableGUI that allows modifying the details of a selected staff member. Create a popup dialog with input fields for editing name, surname, staff ID, and type-specific information. Implement validation to ensure the edited information is valid before updating.

SOLUTION STEPS:

STEP 1: Add Edit Button to HotelTableGUI.java

IN THE BUTTON PANEL SECTION, ADD:

```
// Add Edit Staff button
JButton editButton = new JButton("Edit Staff");
editButton.addActionListener(e -> {
  int selectedRow = myTable.getSelectedRow();
  if (selectedRow >= 0) {
    int modelRow = myTable.convertRowIndexToModel(selectedRow);
```

```
        HotelStaff staff = list.get(modelRow);
        editStaffDialog(staff, modelRow);
    } else {
        JOptionPane.showMessageDialog(this, "Please select a staff member to edit!", "No Selection", JOptionPane.WARNING_MESSAGE);
    }
});

// Add to button panel
buttonPanel.add(editButton);
```

STEP 2: Add Edit Dialog Method to HotelTableGUI.java

```
// Method to show edit dialog
private void editStaffDialog(HotelStaff staff, int modelRow) {
    JPanel panel = new JPanel();
    panel.setLayout(new BoxLayout(panel, BoxLayout.Y_AXIS));

    // Input fields
    JTextField nameField = new JTextField(staff.getName(), 20);
    JTextField surnameField = new JTextField(staff.getSurname(), 20);
    JTextField staffIDField = new JTextField(staff.getStaffID(), 20);

    panel.add(new JLabel("First Name:"));
    panel.add(nameField);
    panel.add(new JLabel("Surname:"));
    panel.add(surnameField);
    panel.add(new JLabel("Staff ID:"));
    panel.add(staffIDField);

    // Type-specific fields
```

```
    JTextField specificField = new JTextField(20);
    JLabel specificLabel = new JLabel();

    if (staff instanceof Manager) {
        specificLabel.setText("License Number:");
        specificField.setText(((Manager) staff).getLicenseNumber());
    } else if (staff instanceof HouseKeeper) {
        specificLabel.setText("Years of Experience:");
        specificField.setText(String.valueOf(((HouseKeeper) staff).getYearsOfExperience()));
    } else if (staff instanceof Receptionist) {
        specificLabel.setText("Languages Spoken:");
        specificField.setText(((Receptionist) staff).getLanguagesSpoken());
    }

    panel.add(specificLabel);
    panel.add(specificField);

    // Show dialog
    int result = JOptionPane.showConfirmDialog(this, panel, "Edit Staff Member",
JOptionPane.OK_CANCEL_OPTION);

    if (result == JOptionPane.OK_OPTION) {
        // Validate input
        String newName = nameField.getText().trim();
        String newSurname = surnameField.getText().trim();
        String newStaffID = staffIDField.getText().trim();
        String newSpecific = specificField.getText().trim();

        if (newName.isEmpty() || newSurname.isEmpty() || newStaffID.isEmpty()) {
```

```
        JOptionPane.showMessageDialog(this, "All fields must be filled!", "Validation Error",
JOptionPane.ERROR_MESSAGE);
        return;
    }

    // Update the staff object
    staff.setName(newName);
    staff.setSurname(newSurname);
    staff.setStaffID(newStaffID);

    // Update type-specific information
    if (staff instanceof Manager && !newSpecific.isEmpty()) {
        ((Manager) staff).setLicenseNumber(newSpecific);
    } else if (staff instanceof HouseKeeper && !newSpecific.isEmpty()) {
        try {
            int years = Integer.parseInt(newSpecific);
            ((HouseKeeper) staff).setYearsOfExperience(years);
        } catch (NumberFormatException ex) {
            JOptionPane.showMessageDialog(this, "Years of experience must be a number!",
"Validation Error", JOptionPane.ERROR_MESSAGE);
            return;
        }
    } else if (staff instanceof Receptionist && !newSpecific.isEmpty()) {
        ((Receptionist) staff).setLanguagesSpoken(newSpecific);
    }

    // Refresh the table
    tableModel.fireTableDataChanged();
    JOptionPane.showMessageDialog(this, "Staff member updated successfully!",
"Success", JOptionPane.INFORMATION_MESSAGE);
```

```
    }
}
```

STEP 3: Add Required Imports to HotelTableGUI.java

```
import javax.swing.BoxLayout;
import javax.swing.JLabel;
import javax.swing.JTextField;
```

Q3.13: Add Guest List Display (DATA MANAGEMENT)

Question: Create a printGuestList() method similar to printHotelStaffList() that displays all guests in a formatted console output. Add this as a new menu option. Include sorting options to display guests by room number, name, or nights stayed.

SOLUTION STEPS:

STEP 1: Add printGuestList() Method to WestminsterHotelManager.java

```
// Method to print guest list
public void printGuestList() {
    if (!hotelGuestList.isEmpty()) {
        System.out.println("\n=== HOTEL GUEST LIST ===");
        System.out.println("Total Guests: " + hotelGuestList.size());
        System.out.println("==========================");
        for (Guest guest : hotelGuestList) {
            System.out.println(guest.toString());
        }
        System.out.println("==========================");
    } else {
        System.out.println("There are no guests in the system.");
    }
}
```

STEP 2: Add Sorting Methods

```
// Method to sort and print guests by room number
```

```
public void printGuestListByRoom() {
   if (!hotelGuestList.isEmpty()) {
      // Create a copy to avoid modifying original list
      ArrayList<Guest> sortedGuests = new ArrayList<>(hotelGuestList);

      // Sort by room number
      sortedGuests.sort((g1, g2) -> Integer.compare(g1.getRoomNumber(),
g2.getRoomNumber()));

      System.out.println("\n=== GUEST LIST (Sorted by Room Number) ===");
      for (Guest guest : sortedGuests) {
         System.out.println(guest.toString());
      }
   } else {
      System.out.println("There are no guests in the system.");
   }
}

// Method to sort and print guests by name
public void printGuestListByName() {
   if (!hotelGuestList.isEmpty()) {
      ArrayList<Guest> sortedGuests = new ArrayList<>(hotelGuestList);

      // Sort by surname, then by name
      sortedGuests.sort((g1, g2) -> {
         int surnameCompare = g1.getSurname().compareToIgnoreCase(g2.getSurname());
         if (surnameCompare == 0) {
            return g1.getName().compareToIgnoreCase(g2.getName());
         }
```

```
            return surnameCompare;
        });

        System.out.println("\n=== GUEST LIST (Sorted by Name) ===");
        for (Guest guest : sortedGuests) {
            System.out.println(guest.toString());
        }
    } else {
        System.out.println("There are no guests in the system.");
    }
}

// Method to sort and print guests by nights stayed
public void printGuestListByNights() {
    if (!hotelGuestList.isEmpty()) {
        ArrayList<Guest> sortedGuests = new ArrayList<>(hotelGuestList);

        // Sort by nights stayed (descending - longest stays first)
        sortedGuests.sort((g1, g2) -> Integer.compare(g2.getNightsStayed(),
g1.getNightsStayed()));

        System.out.println("\n=== GUEST LIST (Sorted by Nights Stayed) ===");
        for (Guest guest : sortedGuests) {
            System.out.println(guest.toString());
        }
    } else {
        System.out.println("There are no guests in the system.");
    }
}
```

STEP 3: Add Menu Options

UPDATE runMenu() METHOD: ADD TO MENU DISPLAY:

```
System.out.println("To Print guest list, press 9");
System.out.println("To Print guest list by room, press 10");
System.out.println("To Print guest list by name, press 11");
System.out.println("To Print guest list by nights, press 12");
```

ADD TO SWITCH STATEMENT:

```
case 9:
  this.printGuestList();
  break;
case 10:
  this.printGuestListByRoom();
  break;
case 11:
  this.printGuestListByName();
  break;
case 12:
  this.printGuestListByNights();
  break;
```

STEP 4: Enhanced Guest Statistics Method

```
// Method to show guest statistics
public void printGuestStatistics() {
  System.out.println("\n=== GUEST STATISTICS ===");
  System.out.println("Total Guests: " + hotelGuestList.size());

  if (!hotelGuestList.isEmpty()) {
    // Calculate total nights
    int totalNights = 0;
```

```
        int minNights = Integer.MAX_VALUE;
        int maxNights = Integer.MIN_VALUE;

        for (Guest guest : hotelGuestList) {
            int nights = guest.getNightsStayed();
            totalNights += nights;
            if (nights < minNights) minNights = nights;
            if (nights > maxNights) maxNights = nights;
        }

        double averageNights = (double) totalNights / hotelGuestList.size();

        System.out.printf("Total Nights: %d%n", totalNights);
        System.out.printf("Average Nights per Guest: %.1f%n", averageNights);
        System.out.printf("Shortest Stay: %d nights%n", minNights);
        System.out.printf("Longest Stay: %d nights%n", maxNights);
        System.out.printf("Total Revenue (£50/night): £%.2f%n", totalNights * 50.0);
    }
    System.out.println("========================");
}
```